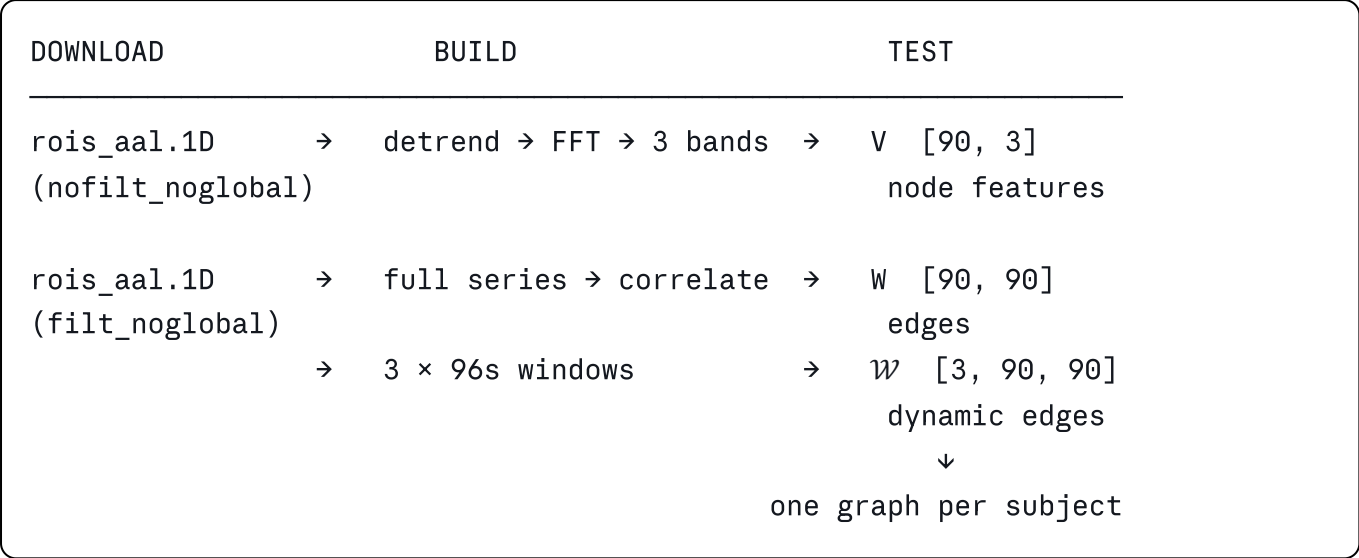


Complete Plan — Every Step

THE FLOW (read this first)



Two downloads. Three objects. That's the whole data pipeline.

You do NOT need: func_preproc, DPARSF, SPM12, MATLAB. All gone.

WHY THIS CHANGED

A-GCL computes ALFF from the **ROI-averaged time series**. You computed ALFF at every **voxel** and then averaged those values.

ALFF involves `|FFT|`, which is nonlinear, so:

average(ALFF(voxels)) ≠ ALFF(average(voxels))

Averaging the signal first cancels noise between voxels, then measures amplitude.
Averaging ALFF values second measures noise-inflated amplitudes, then averages them.
The second is noisier.

`rois_aal.1D` is already the ROI-averaged time series — exactly A-GCL's input.

PHASE 1 — REBUILD ALFF (laptop, no GPU, ~1 day)

Step 1.1 — Download

From ABIDE PCP, C-PAC pipeline:

File	Strategy	Why
rois_aal.1D	nofilt_noglobal	ALFF needs the full spectrum
rois_aal.1D	filt_noglobal	you already have this — keep it

Same 979 subjects. The `.1D` files are small text files.

Step 1.2 — Write the ALFF script

```
python

import numpy as np
from scipy.signal import detrend

BANDS = [(0.010, 0.027),    # slow-5
         (0.027, 0.073),    # slow-4
         (0.010, 0.080)]    # classical

def compute_alff(ts, tr):
    """ts: [T, 90] ROI-mean time series (unfiltered). Returns [90, 3]."""
    ts = detrend(ts, axis=0)          # remove linear drift
    n = ts.shape[0]
    nfft = 2 ** int(np.ceil(np.log2(n)))    # zero-pad, matches DPABI
    amp = 2 * np.abs(np.fft.rfft(ts, n=nfft, axis=0)) / n
    freqs = np.fft.rfftfreq(nfft, d=tr)

    alff = np.zeros((ts.shape[1], 3))
    for b, (lo, hi) in enumerate(BANDS):
        m = (freqs >= lo) & (freqs <= hi)
        alff[:, b] = amp[m].mean(axis=0)    # mean amplitude in band
    return alff

def to_malff(alff):
    """Normalise by whole-brain mean, per band."""
    return alff / alff.mean(axis=0, keepdims=True)
```

Notes:

- `tr` comes from your `subject_tr.csv`, per subject
- Use the **first 90 columns** (labels < 9001) — drop cerebellum and vermis
- No nuisance regression — C-PAC already applied it

Step 1.3 — Verify on ONE subject first

Run the script on a single subject and check:

Check	Expected
Shape	<code>(90, 3)</code>
All finite	no NaN, no Inf
<code>malff.mean(axis=0)</code>	≈ 1.0 for each band
No zero rows	every ROI has a value

Do not run all 979 until these four pass.

Step 1.4 — Run all subjects

Loop over all 979. Save as a single array `(979, 90, 3)` plus a subject-ID list.

Check: 979 rows, no NaN anywhere, no all-zero ROIs. Log any subject that fails and why.

Step 1.5 — Compare old vs new ALFF

For each ROI and each band, correlate old values against new across subjects.

Result	Meaning	Action
$r > 0.95$	the two methods agree	your old ALFF was fine — go to Phase 2 with it
$r = 0.6 - 0.9$	meaningfully different	continue to 1.6
$r < 0.5$	very different	continue to 1.6, and double-check both

Step 1.6 — THE DECISION POINT: classical ML on both

Same folds, same seed, same classifier. ALFF features only, no PCC.

```
python

X_old = old_alff.reshape(979, 270)      # z-scored per subject per band
X_new = new_alff.reshape(979, 270)      # same normalisation

# StratifiedKFold(5, shuffle=True, random_state=123)
# StandardScaler + LinearSVC with C grid
```

Features	Accuracy	AUC
Old ALFF (voxelwise → averaged)	59%	0.616
New ALFF (ROI-mean → FFT)	?	?

This decides everything downstream, and costs zero GPU time.

Whichever wins is your ALFF from here on.

PHASE 2 — THE GNN LADDER (GPU)

Step 2.0 — Fix the NaN crash FIRST

4 of your 12 Stage B jobs died at hour 3-5 with `Beta(nan, nan)`.

```
python

# TA_encoder.py - clamp() does NOT catch NaN, it passes it through
gamma = torch.nan_to_num(edge_weight.mean(), nan=0.5).clamp(eps, 1 - eps)
```

Also add a divergence check in the training loop: if `model_loss` is NaN, log and stop cleanly instead of crashing hours later.

Step 2.1 — Rebuild the dataset

Write the winning ALFF into `norm_matrix` for every subject. Keep $\mathcal{W}$ and $\mathcal{W}$ exactly as they are — they were never in question.

Delete `processed/data.pt` so the loader rebuilds. (This time it genuinely matters, because `Dataset.py`'s inputs changed.)

Check: load 3 known subjects, assert their tensors match the `.mat` files.

Step 2.2 — Set the anchor

Run once. Every later number is measured against this.

```
--node_feature_mode alff      --emb_dim 128    --num_gc_layers 1
--pooling_type standard      --batch_size 32 --epochs 300
--gamma_mode baseline        --reg_lambda 0.2
--weight_decay 1e-4          --early_stop_patience 10
--eval_interval 5            --seed 123
```

Record: `BestValidationScore`, `BestTestScore`, final `fin_reg`, stop epoch.

Step 2.3 — The rounds (one variable each)

Round	Change from anchor	Jobs	Question answered
1	<code>gamma_mode</code> = <code>paper_literal</code> , then <code>signal_strength</code>	2	does turning attention ON help?
2	<code>L=2 standard</code> · <code>L=1 layerwise</code> · <code>L=2 layerwise</code>	3	which architecture?
3	<code>emb_dim</code> 256, 512 at Round 2's winner	2	more capacity?
4	flatten instead of <code>global_add_pool</code> (~5 lines)	2	does ROI identity matter?
5	3 seeds on the overall winner	3	is the winner real or noise?

~12 jobs total.

Round 1 matters most: your anchor uses `gamma=baseline`, which sets $\lambda = 1e-4$ — the paper's attention mechanism has **never been switched on** in any run you've done.

Step 2.4 — Checks on EVERY job

Signal	Healthy	If it fails
<code>fin_reg</code> (drop ratio)	0.2 - 0.5	> 0.7 raise <code>reg_lambda</code> ; < 0.05 lower it
<code>lambda</code> at eval	> 0.1	attention is off — <code>gamma_mode</code> didn't take
Test acc vs "Before training"	above it	training is hurting — stop and diagnose
Stop epoch	< 300	early stopping never fired — check patience

PHASE 3 — FINAL NUMBERS

Step 3.1 — Nested CV check (5 jobs)

Run the winning config through `nested_cv/`. Expect 1-3 points lower than transductive. **That gap is itself a result** — report both.

Step 3.2 — What goes in the paper

Arm	Protocol	Purpose
GNN transductive	5-fold, encoder on all	comparable to GraSTI-ACL and A-GCL
GNN nested	encoder per fold	the honest number
Classical ML	same folds	the baseline to beat
Random encoder	no training	shows what training actually adds

Plus the exclusion cascade: 1035 → 1009 → QC → FD → 979 → final N, with site and diagnosis breakdown.

THE RULES

1. **One variable per round.** Never change two things.
2. **Compare against the anchor.** If a round loses, discard it and keep the anchor.
3. **Pick winners by** `BestValidationScore`, report their `BestTestScore`. Both are in every log. Sorting by test score is test-peeking.
4. **Any jump over +5 points → shuffle the labels and rerun.** A clean pipeline collapses to ~50%. If it doesn't, something is leaking.
5. **Write down every result** — config, val, test, fin_reg, stop epoch. A grid you can't compare later is a grid you have to run again.

START HERE

Today: Step 1.1 (download) and Step 1.2 (write the script). **Tomorrow:** Steps 1.3-1.6. By the end you'll know whether your ALFF was the problem, and it will have cost you no GPU time at all.

Everything after that is the ladder, and the ladder is ~12 jobs.